

Chapter 6 Extras

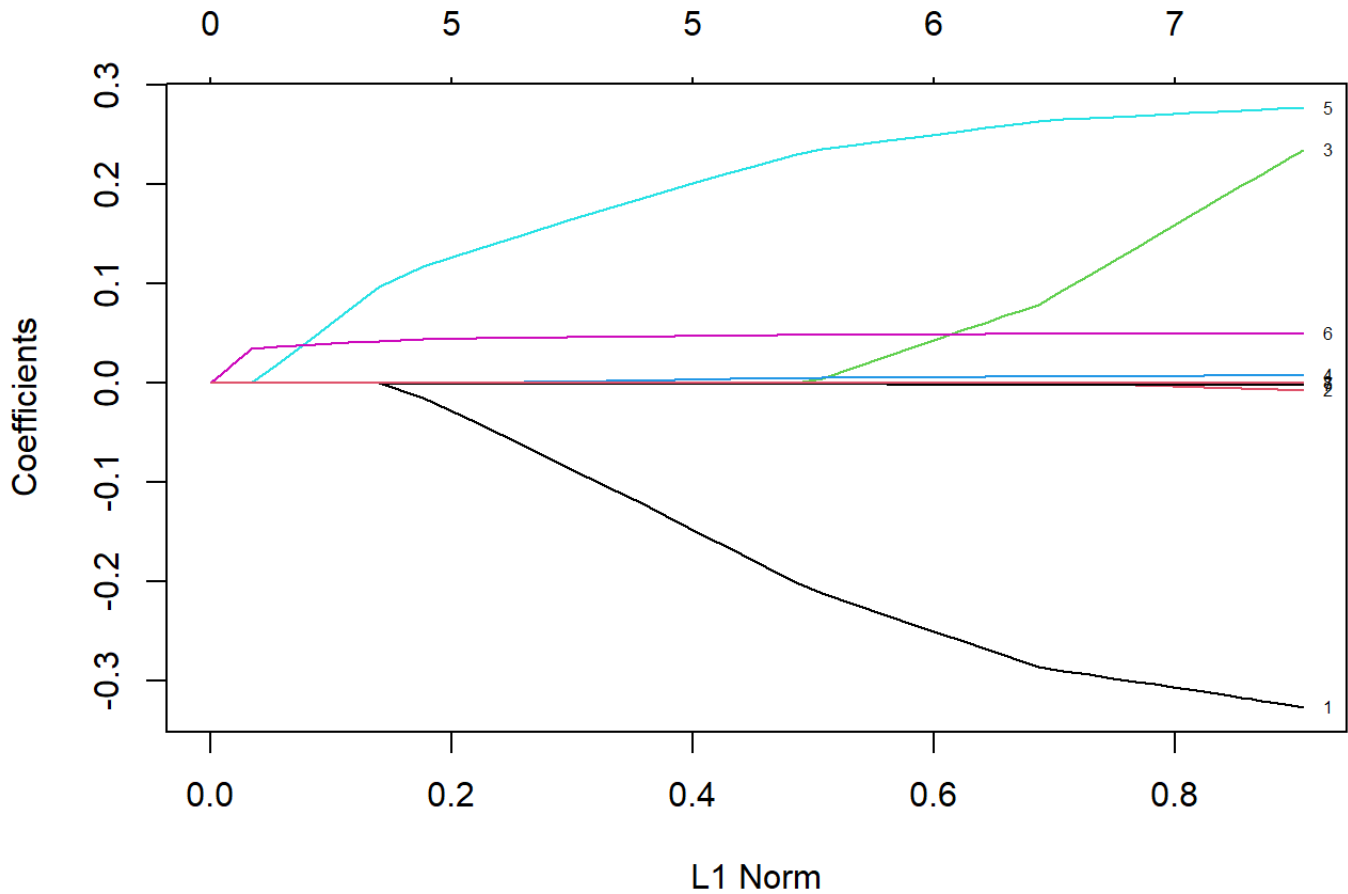
6.1 Using Lasso

```
library(glmnet)

## Loading required package: Matrix

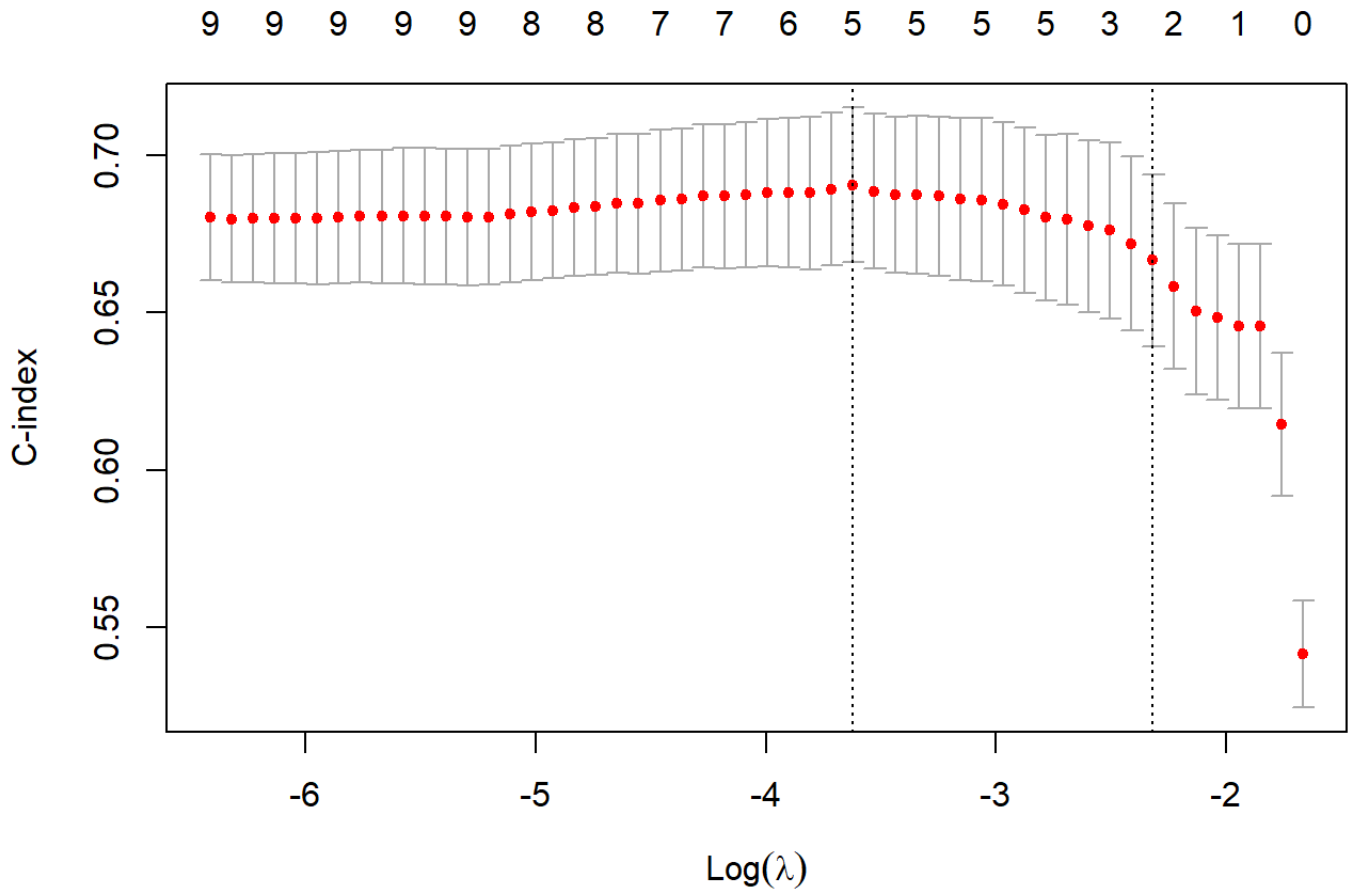
## Loaded glmnet 4.1-7

x=GBSG2 %>% select(horTh,age,menostat,tsize, tgrade, pnodes, progrec, estrec)
y=Surv(time=GBSG2$time,event=GBSG2$cens)
model1<-glmnet(x,y,family="cox")
plot(model1,label=T)
```



```
x1=x %>% mutate(horTh2=if_else(horTh=="no",0,1),menostat2=if_else(menostat=="Pre'
x1<-as.matrix(x1)
set.seed(1287)
cvfit <- cv.glmnet(x1, y, family = "cox", type.measure = "C")
plot(cvfit)

plot(cvfit)
```



```
cvfit$lambda.min
```

```
## [1] 0.02673542
```

```
cvfit$lambda.1se
```

```
## [1] 0.09834302
```

```
coef(model1, s=0.03)
```

```
## 8 x 1 sparse Matrix of class "dgCMatrix"
##              1
## horTh      -0.176785737
## age        .
## menostat   .
## tsize      0.004002226
## tgrade     0.216889004
## pnodes     0.047722334
## progrec    -0.001325963
## estrec     .
```

```
coef(model1, s=0.1)
```

```
## 8 x 1 sparse Matrix of class "dgCMatrix"
##              1
## horTh      .
## age        .
## menostat   .
## tsize      .
## tgrade     0.0153489911
## pnodes     0.0362199648
## progrec    -0.0001341727
## estrec     .
```

Uses Breslow for ties (not able to do Efron...yet)

Can also do repeated events and stratified analysis, see

<https://glmnet.stanford.edu/articles/Coxnet.html>.

6.2 Decision Trees

```
library(rpart)
library(partykit)
```

```
## Warning: package 'partykit' was built under R version 4.3.2
```

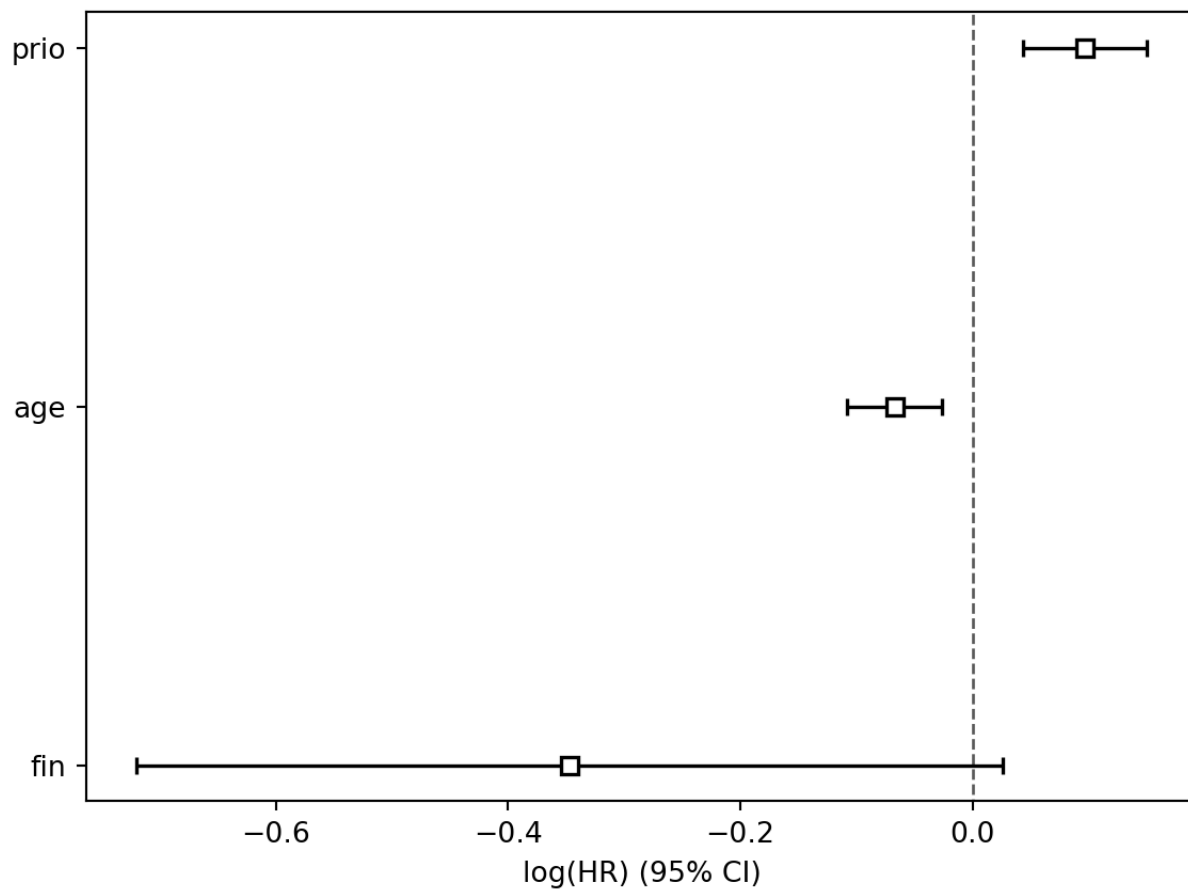
```
## Loading required package: grid
```

```
## Loading required package: libcoin
```

```
## Loading required package: mvtnorm
```

```
tree.surv<- ctree(Surv(time,cens) ~ . ,data=GBSG2)
tree.surv
```

```
##  
## Model formula:  
## Surv(time, cens) ~ horTh + age + menostat + tsize + tgrade +  
##      pnodes + progrec + estrec  
##  
## Fitted party:  
## [1] root  
## |   [2] pnodes <= 3  
## |   |   [3] horTh in no: 2093.000 (n = 248)  
## |   |   [4] horTh in yes: Inf (n = 128)  
## |   [5] pnodes > 3  
## |   |   [6] progrec <= 20: 624.000 (n = 144)  
## |   |   [7] progrec > 20: 1701.000 (n = 166)  
##  
## Number of inner nodes:    3  
## Number of terminal nodes: 4  
  
plot\(tree.surv\)
```



```
predict(tree.surv, newdata = GBSG2[1:2,], type = "node")
```

```
## 1 2
```

```
## 3 7
```

```
predict(tree.surv, newdata = GBSG2[1:2,], type = "response")
```

```
##      1      2
```

```
## 2093 1701
```

This is using conditional trees in the partykit package in R. Good reference for more information is <https://cran.r-project.org/web/packages/partykit/vignettes/ctree.pdf>.

6.3 Random Forest

```
library(randomForestSRC)
```

```
## Warning: package 'randomForestSRC' was built under R version 4.3.2
```

```
##
```

```
## randomForestSRC 3.2.2
```

```
##
```

```
## Type rfsrc.news() to see new features, changes, and bug fixes.
```

```
##
```

```
##
```

```
## Attaching package: 'randomForestSRC'
```

```
## The following object is masked from 'package:Hmisc':
```

```
##
```

```
## impute
```

```
library(ggRandomForests)
```

```
## Warning: package 'ggRandomForests' was built under R version 4.3.2
```



```
## Loading required package: randomForest
```

```
## Warning: package 'randomForest' was built under R version 4.3.2
```

```
## randomForest 4.7-1.1
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
```

```
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

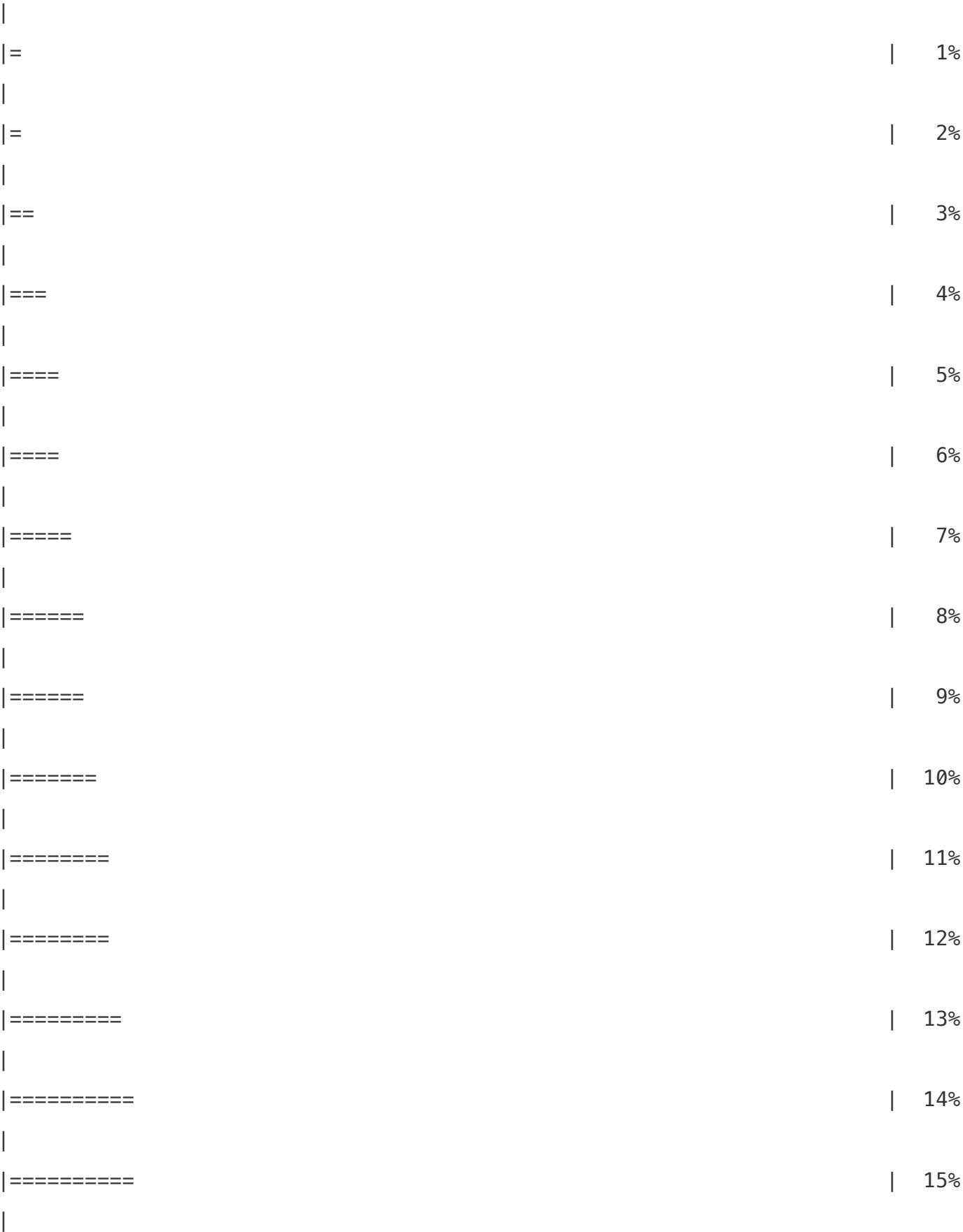
```
##      combine
```

```
## The following object is masked from 'package:ggplot2':
```

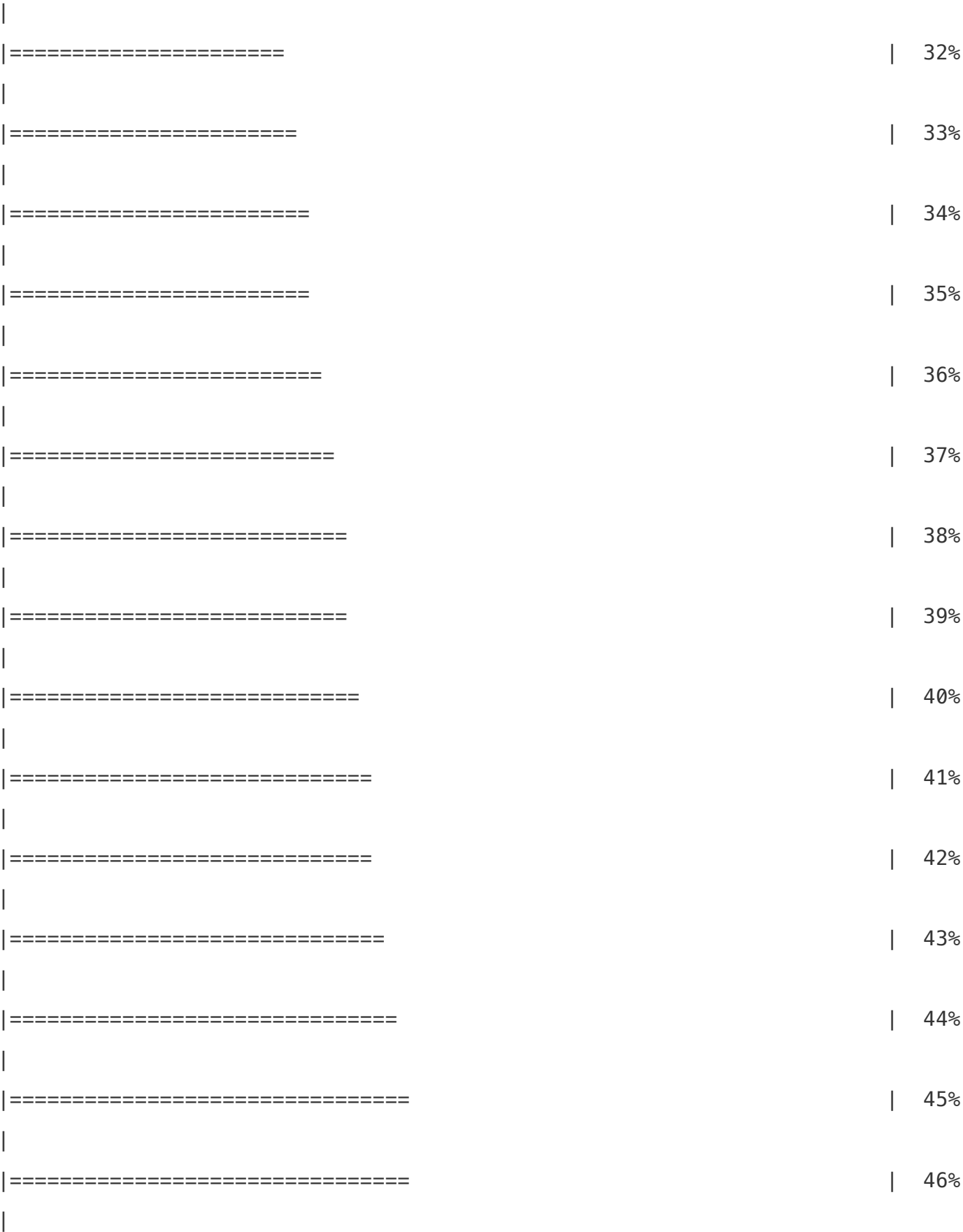
```
##
```

```
##      margin
```

```
surv.rf <- rfsrc(Surv(time,cens) ~ . ,data=GBSG2,importance = TRUE,splitrule="logit")
print(surv.rf$importance)
```

=====	16%
=====	17%
=====	18%
=====	19%
=====	20%
=====	21%
=====	22%
=====	23%
=====	24%
=====	25%
=====	26%
=====	27%
=====	28%
=====	29%
=====	30%
=====	31%



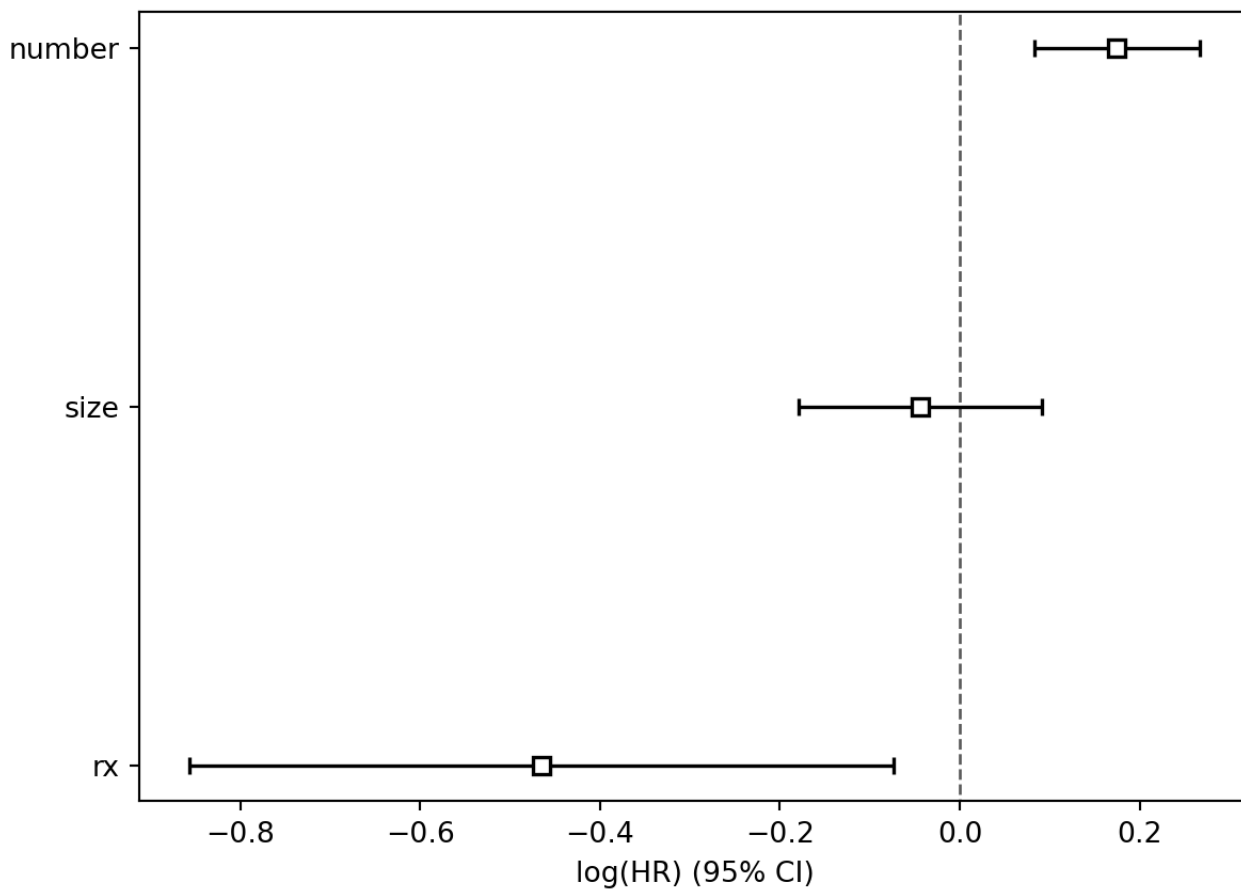


=====		63%
=====		64%
=====		65%
=====		66%
=====		67%
=====		68%
=====		69%
=====		70%
=====		71%
=====		72%
=====		73%
=====		74%
=====		75%
=====		76%
=====		77%

=====	78%
=====	79%
=====	80%
=====	81%
=====	82%
=====	83%
=====	84%
=====	85%
=====	86%
=====	87%
=====	88%
=====	89%
=====	90%
=====	91%
=====	92%
=====	93%



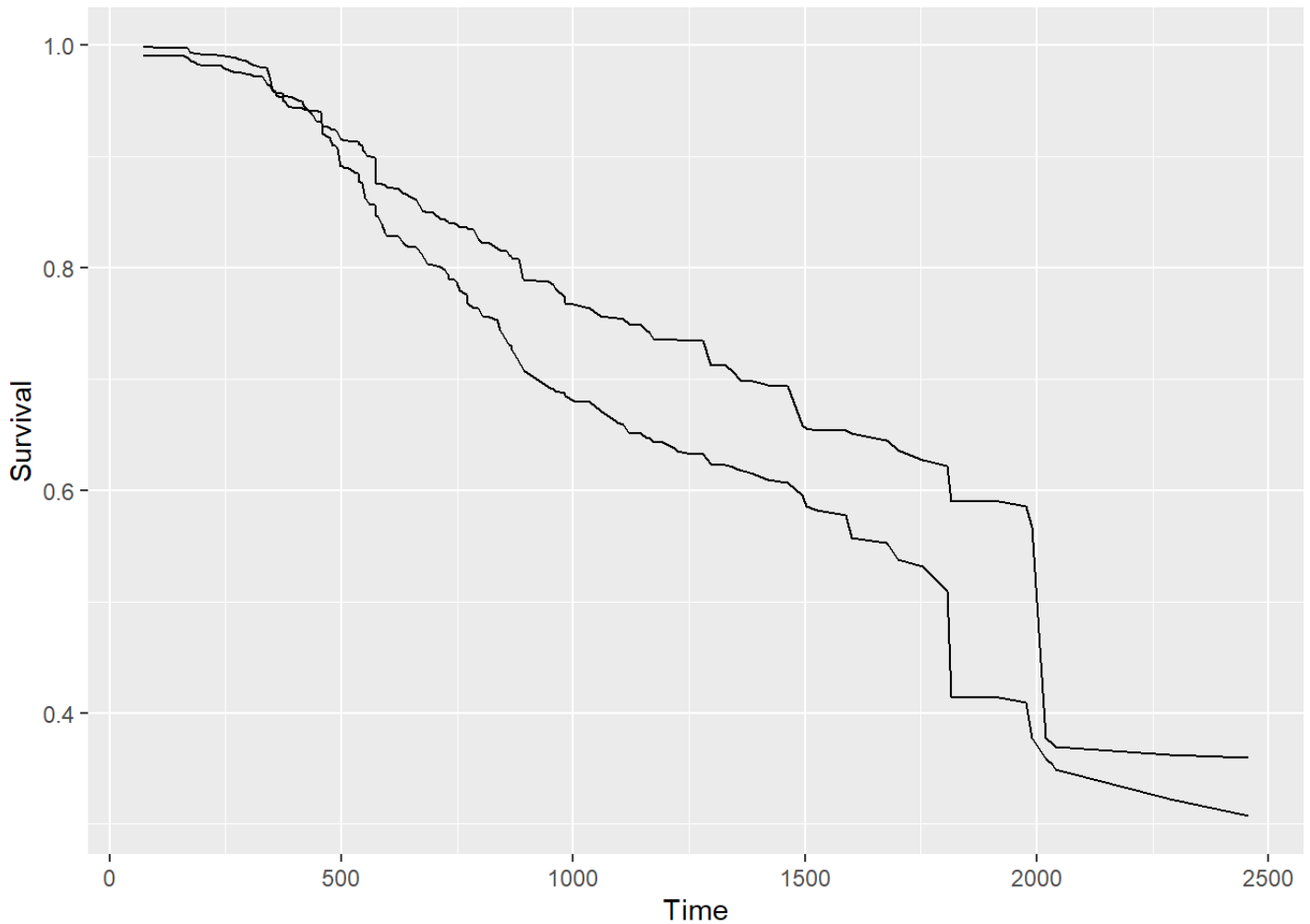
```
par(cex.axis = 2.0, cex.lab = 2.0, cex.main = 2.0, mar = c(6.0,17,1,1), mgp = c(4,1,1),
plot(imp.val, xlab = "Variable Importance (x 100)", cex = 1.2)
```



```

newdat1=GBSG2[1:2,-c(9:10)]
pred.surv=predict(surv.rf,newdata=newdat1)
graph.surv=cbind.data.frame(c(rep(1,length(pred.surv$survival[1,])),rep(2,length(
colnames(graph.surv)=c("Person","Time","Survival")
ggplot(graph.surv,aes(x=Time,y=Survival,group=Person))+geom_line()

```



For more information, see <https://www.randomforestsrc.org/articles/survival.html>, <https://www.randomforestsrc.org/index.html> for the Survival Random Forest or <https://cran.microsoft.com/snapshot/2014-12-19/web/packages/ggRandomForests/vignettes/randomForestSurvival.pdf> for nice visuals using the ggRandomForest package.

6.4 Python

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

```

from sklearn import set_config
from sklearn.preprocessing import OrdinalEncoder

from sksurv.datasets import load_gbsg2
from sksurv.ensemble import RandomSurvivalForest
from sksurv.linear_model import CoxPHSurvivalAnalysis, CoxnetSurvivalAnalysis
from sksurv.preprocessing import OneHotEncoder

set_config(display="text")

X, y = load_gbsg2()

def plot_coefficients(coefs, n_highlight):
    _, ax = plt.subplots(figsize=(9, 6))
    n_features = coefs.shape[0]
    alphas = coefs.columns
    for row in coefs.itertuples():
        ax.semilogx(alphas, row[1:], ".-", label=row.Index)

    alpha_min = alphas.min()
    top_coefs = coefs.loc[:, alpha_min].map(abs).sort_values().tail(n_highlight)
    for name in top_coefs.index:
        coef = coefs.loc[name, alpha_min]
        plt.text(alpha_min, coef, name + " ", horizontalalignment="right", ver

    ax.yaxis.set_label_position("right")
    ax.yaxis.tick_right()
    ax.grid(True)
    ax.set_xlabel("alpha")

```

```
ax.set_ylabel("coefficient")

grade_str = X.loc[:, "tgrade"].astype(object).values[:, np.newaxis]
grade_num = OrdinalEncoder(categories=[["I", "II", "III"]]).fit_transform(grade_str)

X_no_grade = X.drop("tgrade", axis=1)
Xt = OneHotEncoder().fit_transform(X_no_grade)
Xt.loc[:, "tgrade"] = grade_num
```

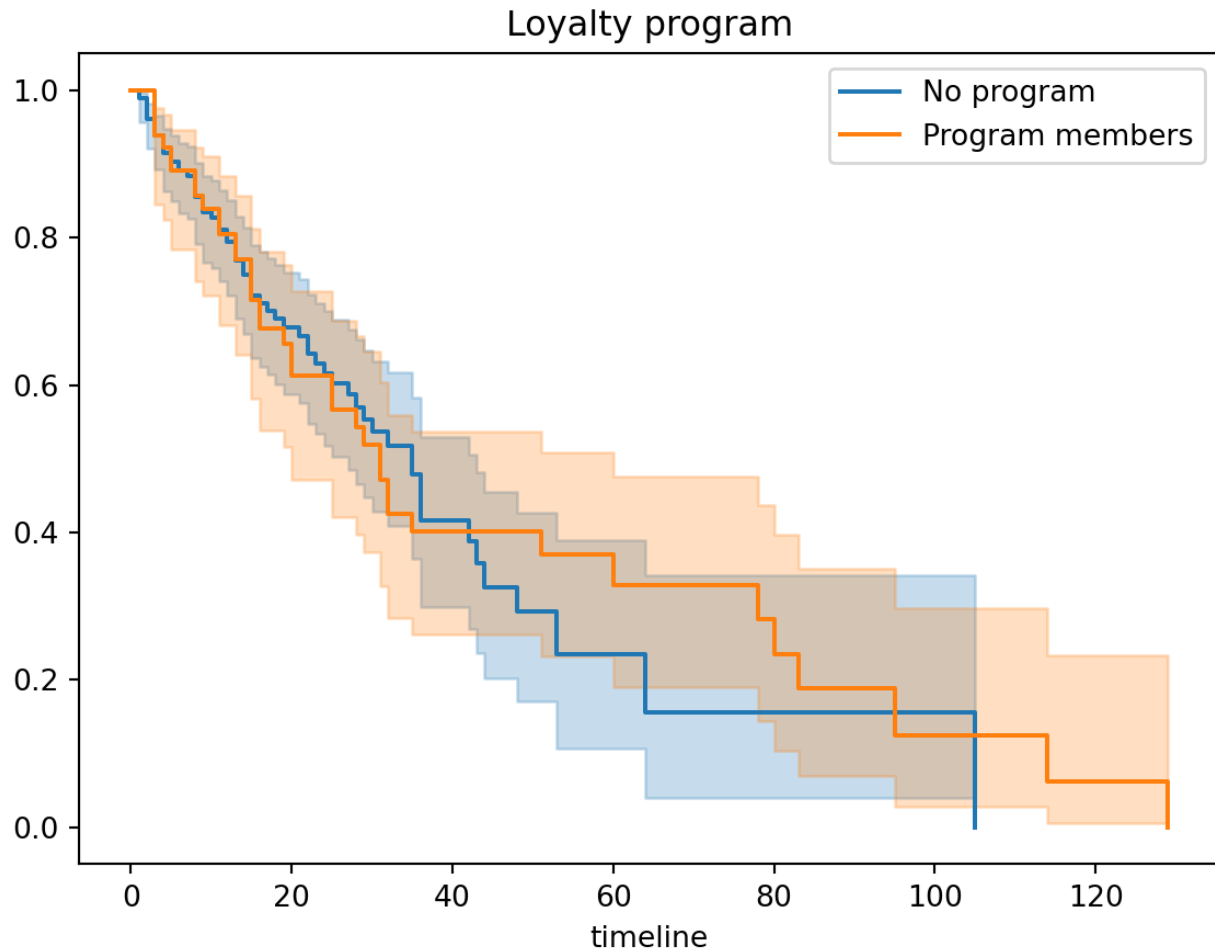
###Lasso

```
cox_lasso = CoxnetSurvivalAnalysis(l1_ratio=1.0, alpha_min_ratio=0.01)
cox_lasso.fit(Xt, y)

## CoxnetSurvivalAnalysis(alpha_min_ratio=0.01, l1_ratio=1.0)

coefficients_lasso = pd.DataFrame(cox_lasso.coef_, index=Xt.columns, columns=np.i

plot_coefficients(coefficients_lasso, n_highlight=5)
plt.show()
```



Random Survival Forest

```
rsf = RandomSurvivalForest(
    n_estimators=1000, min_samples_split=10, min_samples_leaf=15, n_jobs=-1, random_state=23987)
rsf.fit(Xt, y)
```

```
## RandomSurvivalForest(min_samples_leaf=15, min_samples_split=10,
##                        n_estimators=1000, n_jobs=-1, random_state=23987)
```

```
rsf.score(Xt, y)

## 0.76956083924492

surv_plot=Xt.head(2)

surv=rsf.predict_survival_function(surv_plot,return_array=True)

surv

## array([[1.          , 1.          , 1.          , ..., 0.31035269, 0.31035269,
##          0.31035269],
##        [1.          , 1.          , 1.          , ..., 0.31250991, 0.31250991,
##          0.31250991]])

for i, s in enumerate(surv):
    plt.step(rsf.unique_times_, s, where="post")
plt.ylabel("Cumulative hazard")
plt.xlabel("Time in days")
plt.legend()
plt.grid(True)
plt.show()
```

